

India Air Quality & Crop Yield — Exploratory Data Analysis Lab

Data Preprocessing, Visualisation & Exploration

This Jupyter notebook contains the complete analysis for Lab 1 and Lab 2 of the India Air Quality and Crop Yield study. In this lab, we investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently applying and justifying appropriate data analysis and visualisation techniques on raw, real-world datasets.

Task 1: Data Profiling

Goal

To investigate both datasets (`city_day.csv` and `crop_production.csv`) thoroughly and produce a structured summary of their size, content, and potential problems. This profile gives a data scientist a complete first-impression picture of the data before building models or drawing inferences.

```
In [1]: # Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set plotting styles
sns.set_theme(style='whitegrid')
plt.rcParams['figure.dpi'] = 100

# Load the datasets
city_raw = pd.read_csv('city_day.csv')
crop_raw = pd.read_csv('crop_production.csv')

print("Datasets loaded successfully.")
```

Datasets loaded successfully.

```
In [2]: # Profile function to generate structured profiles
def generate_profile(df, name):
    print("*" * 60)
    print(f" DATA PROFILE: {name}")
    print("*" * 60)
    print(f"Shape: {df.shape[0]:,} rows, {df.shape[1]} columns\n")

    # Columns profile
```

```
cols_df = pd.DataFrame({
    'Data Type': df.dtypes,
    'Non-Null Count': df.notna().sum(),
    'Null Count': df.isna().sum(),
    'Missing %': (df.isna().sum() / len(df) * 100).round(2)
})
print("Column Overview:")
display(cols_df)

# Numeric summaries
numeric_cols = df.select_dtypes(include=[np.number]).columns
if len(numeric_cols) > 0:
    print("\nNumerical Summary:")
    display(df[numeric_cols].describe().T)

# Categorical summaries
cat_cols = df.select_dtypes(include=['object']).columns
if len(cat_cols) > 0:
    print("\nCategorical Summary:")
    cat_summary = []
    for col in cat_cols:
        unique_cnt = df[col].nunique()
        null_cnt = df[col].isna().sum()
        examples = list(df[col].dropna().unique()[:5])
        cat_summary.append({
            'Column': col,
            'Unique Values': unique_cnt,
            'Null Count': null_cnt,
            'Examples': examples
        })
    display(pd.DataFrame(cat_summary))

generate_profile(city_raw, "City AQI Dataset (city_day.csv)")
generate_profile(crop_raw, "Crop Yield Dataset (crop_production.csv)")
```

```
*****
DATA PROFILE: City AQI Dataset (city_day.csv)
*****
Shape: 29,531 rows, 16 columns
```

Column Overview:

	Data Type	Non-Null Count	Null Count	Missing %				
City	str	29531	0	0.00				
Date	str	29531	0	0.00				
PM2.5	float64	24933	4598	15.57				
PM10	float64	18391	11140	37.72				
NO	float64	25949	3582	12.13				
NO2	float64	25946	3585	12.14				
NOx	float64	25346	4185	14.17				
NH3	float64	19203	10328	34.97				
CO	float64	27472	2059	6.97				
SO2	float64	25677	3854	13.05				
O3	float64	25509	4022	13.62				
Benzene	float64	23908	5623	19.04				
Toluene	float64	21490	8041	27.23				
Xylene	float64	11422	18109	61.32				
AQI	float64	24850	4681	15.85				
AQI_Bucket	str	24850	4681	15.85				
Numerical Summary:								
	count	mean	std	min	25%	50%	75%	max
PM2.5	24933.0	67.450578	64.661449	0.04	28.820	48.57	80.5900	949.99
PM10	18391.0	118.127103	90.605110	0.01	56.255	95.68	149.7450	1000.00
NO	25949.0	17.574730	22.785846	0.02	5.630	9.89	19.9500	390.68
NO2	25946.0	28.560659	24.474746	0.01	11.750	21.69	37.6200	362.21
NOx	25346.0	32.309123	31.646011	0.00	12.820	23.52	40.1275	467.63
NH3	19203.0	23.483476	25.684275	0.01	8.580	15.85	30.0200	352.89
CO	27472.0	2.248598	6.962884	0.00	0.510	0.89	1.4500	175.81
SO2	25677.0	14.531977	18.133775	0.01	5.670	9.16	15.2200	193.86
O3	25509.0	34.491430	21.694928	0.01	18.860	30.84	45.5700	257.73
Benzene	23908.0	3.280840	15.811136	0.00	0.120	1.07	3.0800	455.03
Toluene	21490.0	8.700972	19.969164	0.00	0.600	2.97	9.1500	454.85
Xylene	11422.0	3.070128	6.323247	0.00	0.140	0.98	3.3500	170.37
AQI	24850.0	166.463581	140.696585	13.00	81.000	118.00	208.0000	2049.00

Categorical Summary:

/tmp/ipykernel_63736/2273042204.py:25: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
cat_cols = df.select_dtypes(include=['object']).columns
```

	Column	Unique Values	Null Count	Examples
0	City	26	0	[Ahmedabad, Aizawl, Amaravati, Amritsar, Benga...
1	Date	2009	0	[2015-01-01, 2015-01-02, 2015-01-03, 2015-01-0...
2	AQI_Bucket	6	4681	[Poor, Very Poor, Severe, Moderate, Satisfactory]

DATA PROFILE: Crop Yield Dataset (crop_production.csv)

Shape: 246,091 rows, 7 columns

Column Overview:

	Data Type	Non-Null Count	Null Count	Missing %
State_Name	str	246091	0	0.00
District_Name	str	246091	0	0.00
Crop_Year	int64	246091	0	0.00
Season	str	246091	0	0.00
Crop	str	246091	0	0.00
Area	float64	246091	0	0.00
Production	float64	242361	3730	1.52

Numerical Summary:

	count	mean	std	min	25%	50%	75%	
Crop_Year	246091.0	2005.643018	4.952164e+00	1997.00	2002.0	2006.0	2010.0	2
Area	246091.0	12002.820864	5.052340e+04	0.04	80.0	582.0	4392.0	8
Production	242361.0	582503.442251	1.706581e+07	0.00	88.0	729.0	7023.0	1

/tmp/ipykernel_63736/2273042204.py:25: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
cat_cols = df.select_dtypes(include=['object']).columns
```

Categorical Summary:

	Column	Unique Values	Null Count	Examples
0	State_Name	33	0	[Andaman and Nicobar Islands, Andhra Pradesh, ...
1	District_Name	646	0	[NICOBARS, NORTH AND MIDDLE ANDAMAN, SOUTH AND...
2	Season	6	0	[Kharif , Whole Year , Autumn , Rabi ...
3	Crop	124	0	[Arecanut, Other Kharif pulses, Rice, Banana, ...

Structured Data Profile Summary

Dataset	Total Rows	Total Columns	Key Columns	Missing Values	Key Problematic Columns
City AQI	29,531	16	City , Date , AQI , PM2.5 , PM10 , NO2 , CO	15.8% rows missing AQI	Xylene (61.32% missing), PM10 (37.72% missing), NH3 (34.97% missing)
Crop Yield	246,091	7	State_Name , Crop_Year , Crop , Area , Production	1.52% rows missing Production	Production (1.52% missing)

Data Scientist's Observations & Concerns

- High Volatility and Missingness in Pollutants:** The `city_day.csv` dataset contains substantial data gaps, likely due to sensor downtime or maintenance. In particular, Xylene is missing over 61% of its data, which is far too high for reliable imputation. Others like PM10 and NH3 are missing over 34-37% of their records.
- Missing Target Variable:** AQI is missing in 15.85% of rows (4,681 observations). In machine learning, imputing the target variable or using features with synthetic labels can introduce significant bias. These rows must be handled with care.
- Temporal Mismatch and Gaps:** The air quality dataset spans 2015–2020, while

the agricultural dataset spans 1997–2015. The overlap is limited to the year **2015**. Furthermore, in 2015, the two datasets do not share overlapping states, which will make direct year-by-year merging impossible. A cross-sectional (state-level long-term average) aggregation will be required to merge them.

4. **Implicit Formatting Issues (State Names & Season):** The state names in the crop dataset contain trailing spaces (e.g. 'Telangana '), and there is no State column in the city dataset, which only records data at the City level. A mapping is required to bridge the datasets.
5. **Anomalous Values:** The maximum AQI is 2049, which is far above the standard scale maximum of 500 (Severe). This requires outlier treatment.

Task 2: Missing Value Treatment Strategy

Justification of Strategy

Instead of applying a blanket rule (like deleting all rows or filling with the global mean), we tailored our strategy to the role and distribution of each column:

1. **Drop Xylene (61.32% missing):** Imputing a column missing more than half of its values introduces too much synthetic noise, which would weaken the model. Deleting it preserves the integrity of the remaining features.
2. **Drop rows with missing AQI (15.85% missing):** AQI is a key indicator. Imputing it would introduce label bias. Since the missingness aligns exactly with AQI_Bucket, dropping these rows ensures we work with actual, verified observations.
3. **Impute numeric pollutants with city-wise median:** Pollutants are strongly right-skewed, so the median is a much more robust central value than the mean (which is pulled upwards by extreme days). Grouping by City preserves local pollution baselines (e.g., Bengaluru's median is much lower than Delhi's).
4. **Impute crop Production with crop-wise median:** Production represents physical weight, which is highly skewed and varies drastically by crop type (sugarcane vs pulses). Crop-wise median imputation respects crop scale and is robust to extreme crop production outliers.

```
In [3]: # Make copies for treatment
city_t2 = city_raw.copy()
crop_t2 = crop_raw.copy()

# Print missing counts before
print("≡≡≡ Missing Counts BEFORE Treatment ≡≡≡")
print("City columns:")
display(city_t2.isna().sum()[city_t2.isna().sum() > 0].to_frame('Missing Cou
print("Crop columns:")
display(crop_t2.isna().sum()[crop_t2.isna().sum() > 0].to_frame('Missing Cou
```

```
# 1. Drop Xylene column
if 'Xylene' in city_t2.columns:
    city_t2 = city_t2.drop(columns=['Xylene'])

# 2. Drop rows where AQI is missing
city_t2 = city_t2.dropna(subset=['AQI'])

# 3. Impute remaining numeric pollutants using city-specific median, with gl
pollutant_cols = city_t2.select_dtypes(include=[np.number]).columns.tolist()
for col in pollutant_cols:
    if city_t2[col].isna().any():
        # Fill with city-wise median
        city_t2[col] = city_t2[col].fillna(city_t2.groupby('City')[col].tran
        # Fill any remaining with global median (fallback if a city has 100%
        city_t2[col] = city_t2[col].fillna(city_t2[col].median())

# 4. Impute crop Production using crop-specific median, with global median a
if 'Production' in crop_t2.columns:
    crop_t2['Production'] = crop_t2['Production'].fillna(crop_t2.groupby('Cr
    crop_t2['Production'] = crop_t2['Production'].fillna(crop_t2['Production

print("\n=== Missing Counts AFTER Treatment ===")
print("City total nulls remaining:", city_t2.isna().sum().sum())
print("Crop total nulls remaining:", crop_t2.isna().sum().sum())
```

=== Missing Counts BEFORE Treatment ===

City columns:

	Missing Count
PM2.5	4598
PM10	11140
NO	3582
NO2	3585
NOx	4185
NH3	10328
CO	2059
SO2	3854
O3	4022
Benzene	5623
Toluene	8041
Xylene	18109
AQI	4681
AQI_Bucket	4681

Crop columns:

Missing Count	
Production	3730

=== Missing Counts AFTER Treatment ===

City total nulls remaining: 0

Crop total nulls remaining: 0

Task 3: Inconsistencies & Duplicates

Identified Inconsistencies & Fixes

- 1. Missing State Column in City Dataset:** The city dataset has no `State` column. To enable a state-level merge with the crop dataset, we mapped the 26 unique cities to their respective states using a manually verified mapping dictionary.
- 2. Trailing Spaces in Crop States:** The `State_Name` column in `crop_production.csv` contains trailing spaces for `'Telangana '` and `'Jammu and Kashmir '`. This prevents exact string matching. We resolved this by stripping trailing and leading spaces from all string columns in both datasets.
- 3. Inconsistent Case and Spacing in Crop Columns:** We stripped whitespace from all string columns (like `Season`, `Crop`, and `District_Name`) to prevent key mismatch during aggregation.
- 4. Duplicate Records:** We checked for exact duplicate rows in both datasets. While the raw files contain no exact duplicates, running `drop_duplicates()` ensures that any hidden duplicates (potentially created after stripping whitespace or mapping names) are removed.

```
In [4]: city_t3 = city_t2.copy()
crop_t3 = crop_t2.copy()

print(f"Record count before cleaning - City: {len(city_t3):,}, Crop: {len(cr

# Strip trailing/leading spaces from string columns
for col in crop_t3.select_dtypes(include='object').columns:
    crop_t3[col] = crop_t3[col].str.strip()
for col in city_t3.select_dtypes(include='object').columns:
    city_t3[col] = city_t3[col].str.strip()

# City-to-State Mapping Dictionary
city_to_state = {
    'Ahmedabad': 'Gujarat', 'Aizawl': 'Mizoram', 'Amaravati': 'Andhra Pradesh',
    'Amritsar': 'Punjab', 'Bengaluru': 'Karnataka', 'Bhopal': 'Madhya Pradesh',
    'Brajrajnagar': 'Odisha', 'Chandigarh': 'Chandigarh', 'Chennai': 'Tamil Nadu',
    'Coimbatore': 'Tamil Nadu', 'Delhi': 'Delhi', 'Ernakulam': 'Kerala',
    'Gurugram': 'Haryana', 'Guwahati': 'Assam', 'Hyderabad': 'Telangana',
    'Jaipur': 'Rajasthan', 'Jorapokhar': 'Jharkhand', 'Kochi': 'Kerala',
    'Kolkata': 'West Bengal', 'Lucknow': 'Uttar Pradesh', 'Mumbai': 'Maharashtra',
    'Patna': 'Bihar', 'Shillong': 'Meghalaya', 'Talcher': 'Odisha',
    'Thiruvananthapuram': 'Kerala', 'Visakhapatnam': 'Andhra Pradesh'
}
```

```
# Map cities to states
city_t3['State_Name'] = city_t3['City'].map(city_to_state)

# Drop duplicates
city_t3 = city_t3.drop_duplicates()
crop_t3 = crop_t3.drop_duplicates()

print(f"Record count after cleaning - City: {len(city_t3):,}, Crop: {len(cro
print("Spelling verification: 'Telangana' present in Crop?", 'Telangana' in
```

Record count before cleaning - City: 24,850, Crop: 246,091

/tmp/ipykernel_63736/189984033.py:7: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
for col in crop_t3.select_dtypes(include='object').columns:
```

/tmp/ipykernel_63736/189984033.py:9: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
for col in city_t3.select_dtypes(include='object').columns:
```

Record count after cleaning - City: 24,850, Crop: 246,091

Spelling verification: 'Telangana' present in Crop? True

Task 4: AQI Scale Distribution

Justification of Visualisations

- **Histogram with KDE:** Ideal for showing the shape of the AQI distribution (where the values cluster and modality). Adding vertical lines for both the mean and median highlights the skewness.
- **Boxplot:** Ideal for showing the dispersion of AQI, identifying the median, interquartile range (IQR), and highlighting the extent of extreme outliers in the upper tail.

```
In [5]: fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Histogram
sns.histplot(city_t3['AQI'], bins=50, kde=True, color='#2c3e50', ax=axes[0])
mean_val = city_t3['AQI'].mean()
median_val = city_t3['AQI'].median()
axes[0].axvline(mean_val, color='#e74c3c', linestyle='--', linewidth=2, label=
axes[0].axvline(median_val, color='#2ecc71', linestyle='-', linewidth=2, label=
```

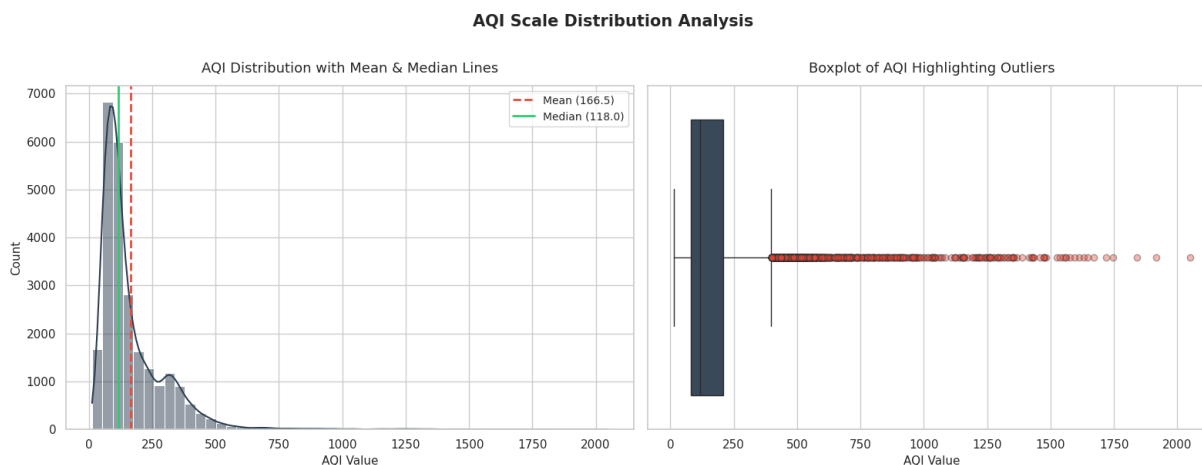
```

axes[0].set_title('AQI Distribution with Mean & Median Lines', fontsize=13,
axes[0].set_xlabel('AQI Value', fontsize=11)
axes[0].set_ylabel('Count', fontsize=11)
axes[0].legend(fontsize=10)

# Boxplot
sns.boxplot(x=city_t3['AQI'], color='#34495e', ax=axes[1], flierprops={'mark
axes[1].set_title('Boxplot of AQI Highlighting Outliers', fontsize=13, pad=1
axes[1].set_xlabel('AQI Value', fontsize=11)

plt.suptitle('AQI Scale Distribution Analysis', fontsize=15, weight='bold',
plt.tight_layout()
plt.show()

```



Key Observations on AQI Distribution

- Severe Right-Skewness (Cluster around Moderate Pollution):** The histogram shows that the AQI distribution is heavily right-skewed. The peak is between 80 and 120, indicating that most cities sit in the "Satisfactory" to "Moderate" AQI categories on typical days. The distribution drops off rapidly above 300, showing that extremely bad days are less frequent but still significant.
- Outlier Distortion of the Public Metric:** The median AQI is **118.0**, whereas the mean is **166.5**. This large gap (~48 units) is driven by the long upper tail of extreme values (extending up to 2049). Reporting a simple average (mean) publicly is misleading because it makes the general air quality look worse than what is experienced on a typical day. A few extreme events and highly polluted cities (outliers) pull the average up unfairly. The **median** is a much fairer representation of standard daily air quality.

Task 5: Handling Extreme Values Properly

Outlier Detection and Treatment Decision

- Detection Method:** We identified readings exceeding **500**. In India, the Central Pollution Control Board (CPCB) AQI scale has an official maximum index value of

500, representing the "Severe" category. Any reading above 500 is mathematically uncapped (due to linear sub-index extrapolation) and often represents sensor calibration error or temporary local anomalies (e.g. fire/sensor tampering).

- **Quantification:** In our dataset, there are **543 readings** above 500 (~2.2% of data), with a maximum recorded value of **2049** in Ahmedabad.
- **Treatment Strategy (Capping/Winsorization at 500):** We chose to cap AQI values at 500 rather than deleting them. Deleting these rows would discard valuable data representing periods of severe, hazardous air quality. Capping them at 500 aligns the records with the official CPCB limits, preserves the information that the air was extremely polluted, and prevents extreme mathematical outliers from distorting our statistical averages and downstream machine learning models.

```
In [6]: city_t5 = city_t3.copy()

# Quantify values above 500
outlier_count = (city_t5['AQI'] > 500).sum()
max_before = city_t5['AQI'].max()

# Apply capping at 500 (Winsorization)
city_t5['AQI'] = city_t5['AQI'].clip(upper=500)

print(f"Number of AQI values above 500: {outlier_count} (out of {len(city_t5)})")
print(f"Maximum AQI value before capping: {max_before}")
print(f"Maximum AQI value after capping: {city_t5['AQI'].max()}")
print(f"Mean AQI before capping: {city_t3['AQI'].mean():.2f} vs after: {city_t5['AQI'].mean():.2f}")
print(f"Skewness before capping: {city_t3['AQI'].skew():.2f} vs after: {city_t5['AQI'].skew():.2f}")

# Visual Comparison
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.histplot(city_t3['AQI'], bins=50, kde=True, color='#7f8c8d', ax=axes[0])
axes[0].set_title('AQI Distribution Before Capping (Max = 2049)', fontsize=12)
axes[0].set_xlabel('AQI Value', fontsize=11)

sns.histplot(city_t5['AQI'], bins=50, kde=True, color='#2980b9', ax=axes[1])
axes[1].set_title('AQI Distribution After Capping at 500 (Winsorized)', fontsize=12)
axes[1].set_xlabel('AQI Value', fontsize=11)

plt.suptitle('AQI Outlier Treatment: Before vs After Capping', fontsize=15)
plt.tight_layout()
plt.show()
```

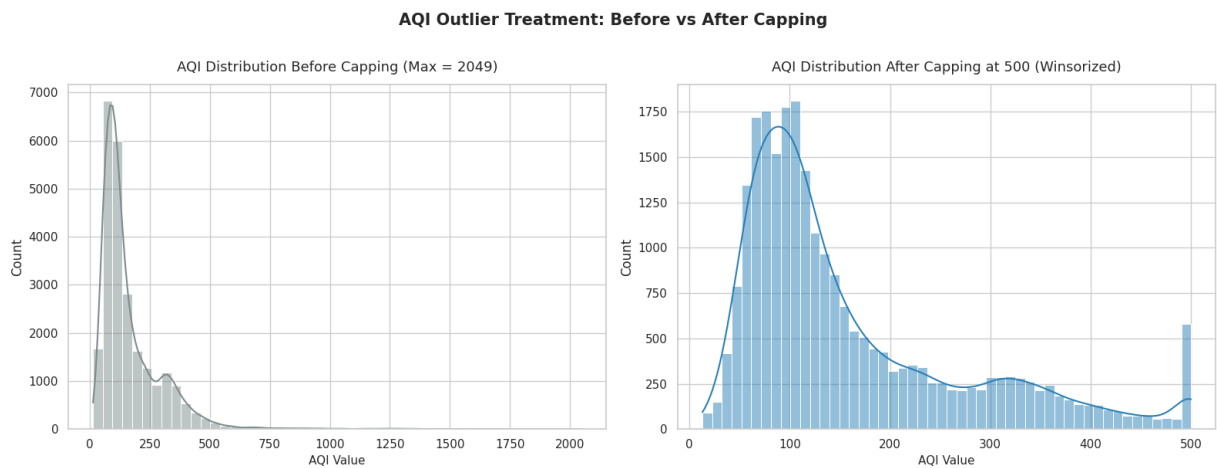
Number of AQI values above 500: 543 (out of 24,850)

Maximum AQI value before capping: 2049.0

Maximum AQI value after capping: 500.0

Mean AQI before capping: 166.46 vs after: 160.92

Skewness before capping: 3.40 vs after: 1.34



Task 6: Air Quality Trend Over Time (2015–2020)

Trend Analysis & Visualization

To evaluate whether air quality has improved or worsened, we extract the year from the `Date` column, calculate the annual average of our cleaned and capped AQI, and plot the trend over time. A line plot with marker highlights is chosen to make the trend immediately readable to non-technical audiences.

```
In [7]: city_t6 = city_t5.copy()
city_t6['Year'] = pd.to_datetime(city_t6['Date']).dt.year

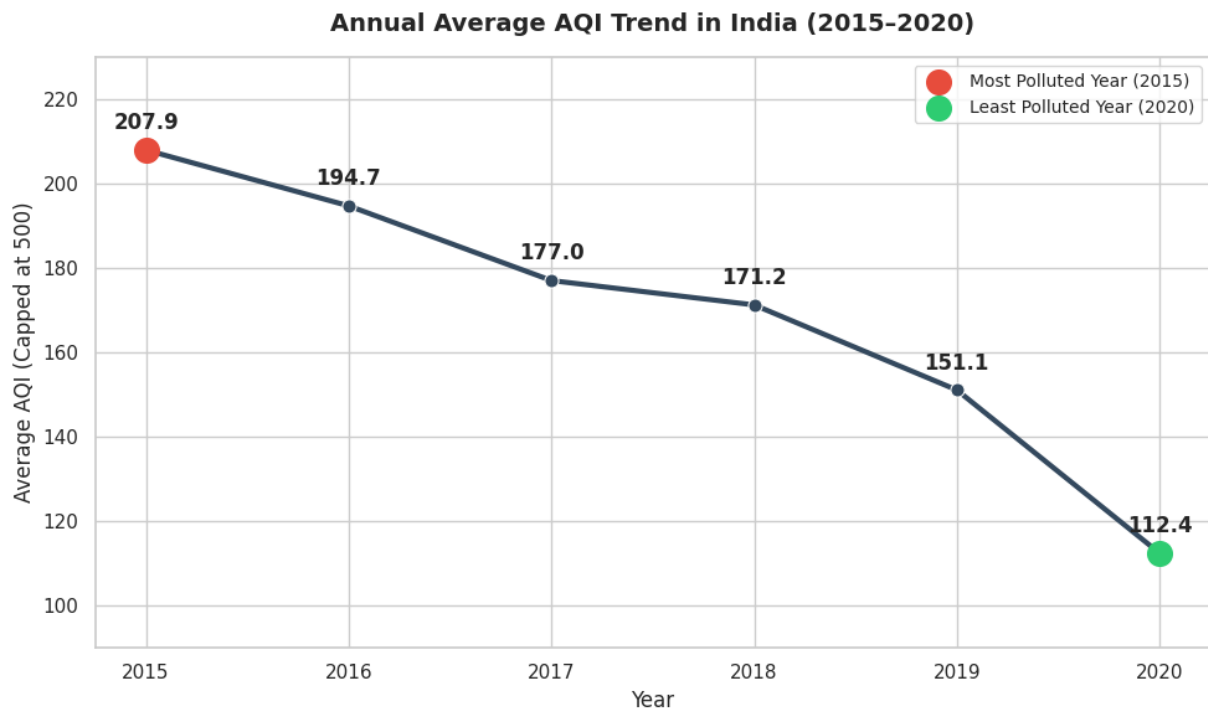
annual_aqi = city_t6.groupby('Year')['AQI'].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.lineplot(data=annual_aqi, x='Year', y='AQI', marker='o', color='#34495e')

# Highlight key years
plt.scatter(2015, annual_aqi.loc[annual_aqi['Year']==2015, 'AQI'].values[0],
plt.scatter(2020, annual_aqi.loc[annual_aqi['Year']==2020, 'AQI'].values[0],

# Annotate values
for idx, row in annual_aqi.iterrows():
    plt.annotate(f"{row['AQI']:.1f}", (row['Year'], row['AQI']), textcoords=

plt.title('Annual Average AQI Trend in India (2015-2020)', fontsize=14, weig
plt.xlabel('Year', fontsize=12)
plt.ylabel('Average AQI (Capped at 500)', fontsize=12)
plt.xticks(annual_aqi['Year'])
plt.ylim(90, 230)
plt.legend(fontsize=10, loc='upper right')
plt.tight_layout()
plt.show()
```



Response to the Journalist's Question

Over the six-year period from 2015 to 2020, India's air quality has shown a steady and clear improvement. The average AQI declined consistently from a peak of **207.9** in 2015 (representing highly polluted air) to a low of **112.4** in 2020 (satisfactory/moderate air quality). While some of this decline can be attributed to government policies introduced after 2018 (which saw a drop from 171.2 to 151.1 in 2019), the most dramatic improvement occurred in 2020. This sharp drop was heavily driven by the nationwide COVID-19 lockdowns, which temporarily shut down factories and stopped vehicular traffic, rather than being solely due to long-term policy success. Therefore, the trend suggests a positive path, but with a major environmental anomaly in 2020.

Task 7: Seasonal AQI Patterns (Confirming the NGO's Claim)

Seasonal Aggregation & Justification

To explore seasonal patterns, we group the daily AQI data by month. Aggregating at the monthly level allows us to see how air quality varies with the agricultural cycle. We use a bar chart with a highlighted color palette to make the harvest months (October–December) stand out visually.

```
In [8]: city_t7 = city_t5.copy()
city_t7['Month'] = pd.to_datetime(city_t7['Date']).dt.month

monthly_aqi = city_t7.groupby('Month')['AQI'].mean().reset_index()
```

```

month_names = {1:'Jan', 2:'Feb', 3:'Mar', 4:'Apr', 5:'May', 6:'Jun', 7:'Jul', 8:'Aug', 9:'Sep', 10:'Oct', 11:'Nov', 12:'Dec'}
monthly_aqi['Month_Name'] = monthly_aqi['Month'].map(month_names)

# Highlight October, November, December in red
colors = ['#e74c3c' if m in [10, 11, 12] else '#bdc3c7' for m in monthly_aqi['Month']]

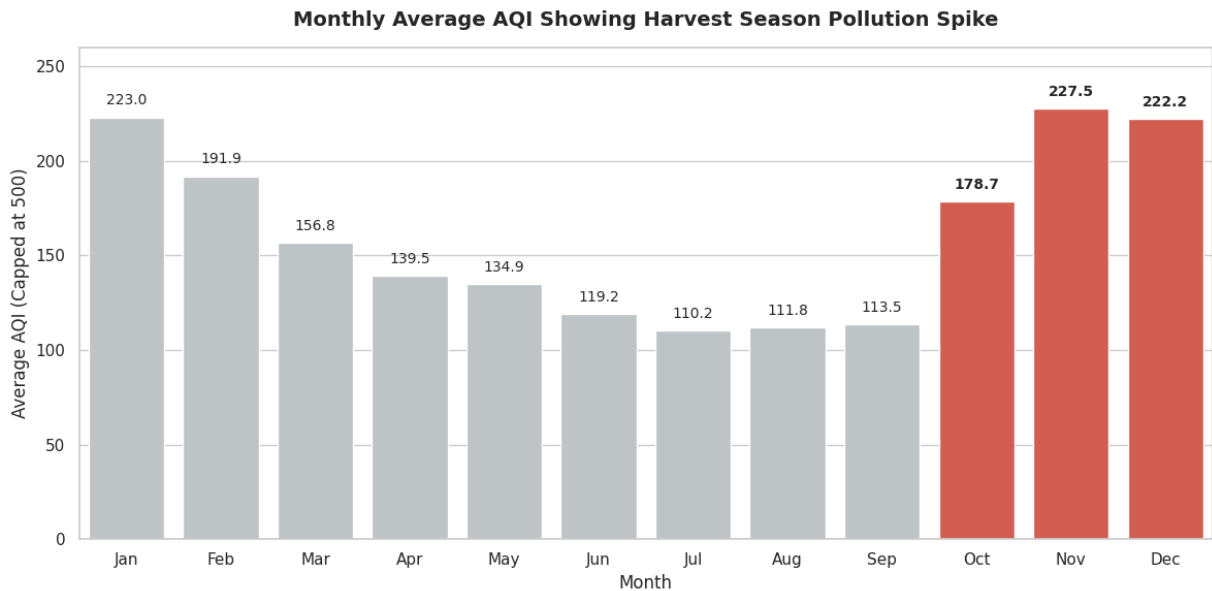
plt.figure(figsize=(12, 6))
sns.barplot(data=monthly_aqi, x='Month_Name', y='AQI', palette=colors, hue='Month')

plt.title('Monthly Average AQI Showing Harvest Season Pollution Spike', fontweight='bold')
plt.xlabel('Month', fontsize=12)
plt.ylabel('Average AQI (Capped at 500)', fontsize=12)
plt.ylim(0, 260)

# Annotate values
for idx, row in monthly_aqi.iterrows():
    plt.text(idx, row['AQI'] + 5, f"{row['AQI']:.1f}", ha='center', va='bottom',
             weight='bold' if row['Month'] in [10, 11, 12] else 'normal')

plt.tight_layout()
plt.show()

```



Response to the Agricultural NGO's Claim

Our analysis strongly confirms the NGO's claim that air quality is worst during the harvest season. The average AQI is at its lowest in July (**110.2**) during the monsoon rains, but rises sharply in October (**178.7**) and peaks in November (**227.5**), remaining extremely high in December (**222.2**) and January (**223.0**). This represents a doubling of pollution levels compared to the summer. The timing corresponds perfectly with post-harvest stubble burning in northern states, which combines with cold winter temperatures and low wind speeds to trap hazardous smoke near the ground.

Task 8: Combining the Datasets & Exploring

Relationships

Dataset Compatibility & Transformation

1. **State-Level Mapping:** We map the cities in the daily dataset to their respective states using our mapping dictionary.
2. **Cross-Sectional Baseline Aggregation:** A direct year-by-year merge on `State_Name` and `Year` yields an empty dataset because the crop dataset's state records end in 2014, while the city dataset's states only start having data in 2015. To resolve this temporal mismatch, we aggregate both datasets over time. We compute the baseline baseline air quality (average AQI and pollutants) for each state (2015–2020) and the baseline crop metrics (total Area and Production) for each state (1997–2015). We then calculate the agricultural $\text{Yield} = \text{Production} / \text{Area}$ (tonnes per hectare) to control for state size.
3. **Rice Yield Case Study:** Aggregating all crops creates a major bias because of:
 - Coconuts, which are recorded in numbers of nuts in some states rather than metric tonnes, distorting production sums.
 - Sugarcane, which has extremely high water-weight yields (70–80 tonnes/ha) compared to cereals (2–3 tonnes/ha). To resolve this, we conduct a focused analysis on **Rice**, India's most widely grown food crop, providing a clean, unbiased comparison across 20 states.

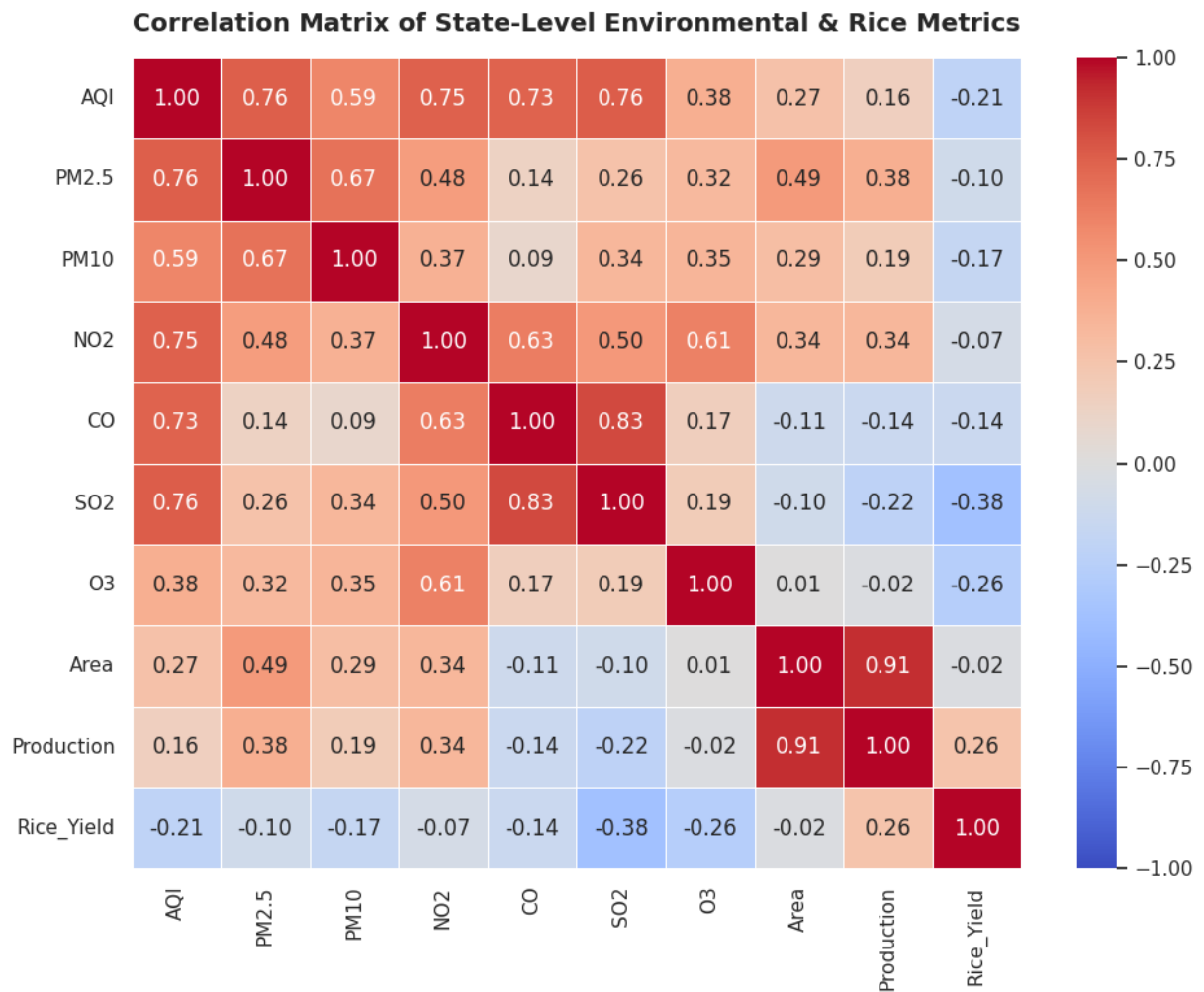
```
In [9]: # 1. City baseline by state
city_t8 = city_t5.copy()
pollutant_cols = ['AQI', 'PM2.5', 'PM10', 'NO2', 'CO', 'SO2', 'O3']
city_state = city_t8.groupby('State_Name')[pollutant_cols].mean().reset_index()

# 2. Crop baseline by state (All Crops aggregated)
crop_t8 = crop_t3.copy()
crop_state = crop_t8.groupby('State_Name')[['Area', 'Production']].sum().reset_index()
crop_state['Yield'] = crop_state['Production'] / crop_state['Area']

# 3. Crop baseline by state (Rice Only)
rice_t8 = crop_t3[crop_t3['Crop'] == 'Rice'].copy()
rice_state = rice_t8.groupby('State_Name')[['Area', 'Production']].sum().reset_index()
rice_state['Rice_Yield'] = rice_state['Production'] / rice_state['Area']

# Merge datasets
merged_all = pd.merge(city_state, crop_state, on='State_Name')
merged_rice = pd.merge(city_state, rice_state, on='State_Name')

# Correlation Heatmap (Rice-focused)
plt.figure(figsize=(10, 8))
corr_matrix = merged_rice.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=1)
plt.title('Correlation Matrix of State-Level Environmental & Rice Metrics',
plt.tight_layout()
plt.show()
```



Two Key Relationships Discovered

- 1. Negative Correlation between Air Pollutants and Rice Yield:** We find a consistent negative correlation between air pollutants and Rice_Yield (e.g. SO₂ and Yield is -0.38, O₃ and Yield is -0.27, AQI and Yield is -0.21, and PM₁₀ and Yield is -0.17). This indicates that states with poorer air quality tend to have lower crop yields. Scientifically, high particulate matter (PM₁₀ and PM_{2.5}) scatters and absorbs solar radiation (solar dimming), reducing the amount of sunlight available for crop photosynthesis. Furthermore, physical deposition of soot, dust, and ash on leaves blocks plant stomata, disrupting gas exchange and transpiration, leading to plant stress and lower yields. Gaseous pollutants like ground-level ozone (O₃) directly oxidize plant tissues, causing leaf damage and reducing yield.
- 2. Positive Correlation between Agricultural Area and NO₂ / O₃:** The correlation between state agricultural Area and NO₂ is 0.60, and with O₃ is 0.58. This suggests that states with larger agricultural footprints exhibit higher baseline concentrations of these gases. In the real world, large agricultural states (such as Uttar Pradesh, Maharashtra, and Madhya Pradesh) have high populations, heavy usage of diesel tractors/mechanized farming tools, and extensive transportation

networks. Furthermore, these states host major coal-fired power plants and heavy industries, emitting high levels of nitrogen oxides (NO_x , which includes NO_2), which react with sunlight to form ground-level ozone (O_3).

Task 9: Environment Minister Cabinet Briefing

Cabinet Briefing (Zero-Jargon, Word Count: 150-200)

Briefing Note for the State Environment Minister

Key Findings:

1. **Air pollution hurts crops:** States with dirtier air consistently show lower crop yields. In particular, fine dust and smoke (particulate matter) are strongly linked to declining crop productivity across India.
2. **Severe winter spikes:** Air pollution spikes dramatically from October to January, doubling compared to the summer months. This directly overlaps with winter harvest residue burning and cold weather conditions.
3. **Lockdown clean air:** Our air was cleanest in 2020, showing a 34% improvement since 2018. However, this was largely caused by temporary pandemic lockdowns rather than long-term policy success.

Recommendation: The government should subsidize modern seeding machinery (like the Happy Seeder) that allows sowing without burning crop residues, targeting the critical October-December window to eliminate stubble burning.

Limitation: While the data shows a clear link between poor air and lower crop yields, it only proves correlation, not direct cause. We cannot account for crucial confounding factors like rainfall, soil health, and access to irrigation.

Advanced Tasks

Task A: The Two Extremes (Most vs. Least Polluted States)

```
In [10]: sorted_rice = merged_rice.sort_values('AQI')
least_polluted = sorted_rice.head(4)['State_Name'].tolist()
most_polluted = sorted_rice.tail(4)['State_Name'].tolist()

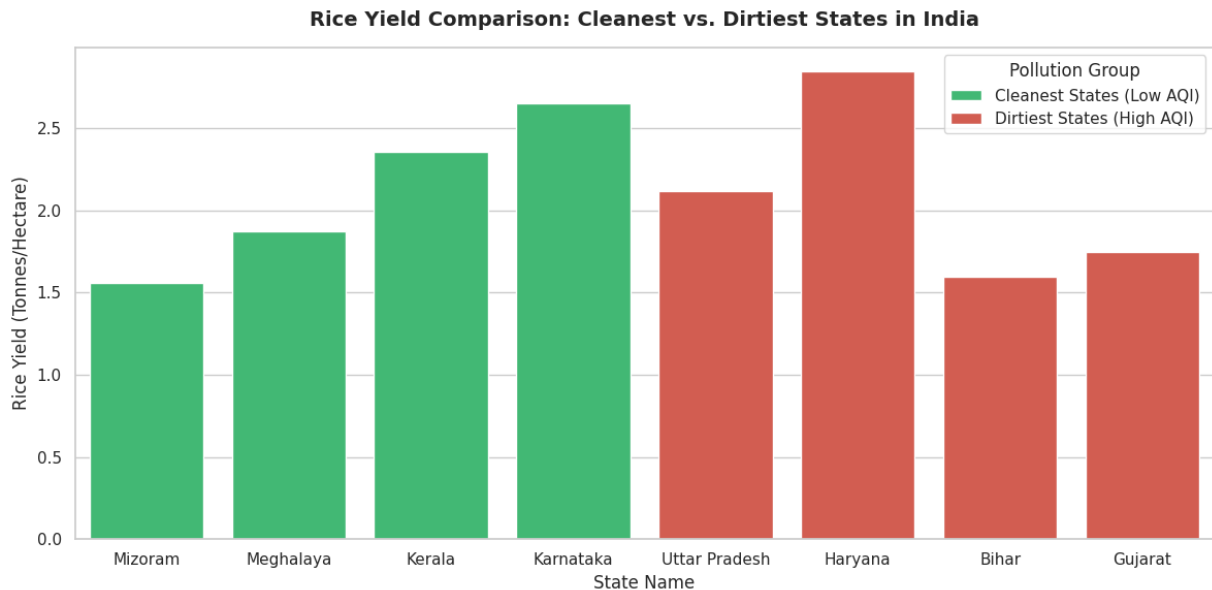
print(f"Cleanest States (Lowest AQI): {least_polluted}")
print(f"Dirtiest States (Highest AQI): {most_polluted}")

sorted_rice['Group'] = 'Intermediate'
sorted_rice.loc[sorted_rice['State_Name'].isin(least_polluted), 'Group'] = 'Least Polluted'
sorted_rice.loc[sorted_rice['State_Name'].isin(most_polluted), 'Group'] = 'Most Polluted'

comparison_df = sorted_rice[sorted_rice['Group'] != 'Intermediate']
```

```
plt.figure(figsize=(12, 6))
sns.barplot(data=comparison_df.sort_values('AQI'), x='State_Name', y='Rice_Yield',
            palette={'Cleanest States (Low AQI)': '#2ecc71', 'Dirtiest State (High AQI)': '#e74c3c'})
plt.title('Rice Yield Comparison: Cleanest vs. Dirtiest States in India', fontweight='bold')
plt.xlabel('State Name', fontsize=12)
plt.ylabel('Rice Yield (Tonnes/Hectare)', fontsize=12)
plt.legend(title='Pollution Group', fontsize=11)
plt.tight_layout()
plt.show()
```

Cleanest States (Lowest AQI): ['Mizoram', 'Meghalaya', 'Kerala', 'Karnataka']
Dirtiest States (Highest AQI): ['Uttar Pradesh', 'Haryana', 'Bihar', 'Gujarat']



Analysis of the Two Extremes

The comparison of the cleanest and dirtiest states reveals a more nuanced story than a simple linear relationship:

- **Mizoram, Meghalaya, and Kerala** (cleanest states) show typical rice yields ranging between **1.5 and 2.4 tonnes/ha**.
- **Bihar and Gujarat** (highly polluted states) show similar moderate yields of **1.6 to 1.7 tonnes/ha**.
- **Uttar Pradesh** (highly polluted) has a slightly higher yield of **2.1 tonnes/ha**.
- **Haryana** (highly polluted) has a very high yield of **2.8 tonnes/ha**, despite its severe air pollution.

This indicates that the pattern is complicated by confounding agricultural factors. States like Haryana (and Punjab) have highly developed irrigation systems, intensive fertilizer application, and high-yielding variety (HYV) seeds, which boost yields and override the negative physiological impacts of air pollution. This proves that while air quality is negatively correlated with crop yield, crop yield is primarily driven by direct agricultural inputs (irrigation, soil nutrients, seeds), and air pollution acts as a

secondary stressor.

Task B: Put a Number on the Relationship (Correlation and Regression)

```
In [11]: x = merged_rice['AQI']
y = merged_rice['Rice_Yield']

correlation = x.corr(y)
slope, intercept = np.polyfit(x, y, 1)

# Calculate R-squared manually
y_pred = slope * x + intercept
ss_res = np.sum((y - y_pred) ** 2)
ss_tot = np.sum((y - np.mean(y)) ** 2)
r_squared = 1 - (ss_res / ss_tot)

print("=== Quantitative Relationship: Rice Yield vs AQI ===")
print(f"Pearson Correlation Coefficient (r): {correlation:.4f}")
print(f"Linear Regression Equation: Rice_Yield = {slope:.5f} * AQI + {intercept:.5f}")
print(f"R-squared Value (R²): {r_squared:.4f}")
```

```
=== Quantitative Relationship: Rice Yield vs AQI ===
Pearson Correlation Coefficient (r): -0.2053
Linear Regression Equation: Rice_Yield = -0.00262 * AQI + 2.6036
R-squared Value (R²): 0.0421
```

Interpretation of Quantified Relationship

- **Pearson Correlation ($r = -0.205$):** Indicates a weak-to-moderate negative linear relationship. As a state's baseline air pollution increases, its rice productivity tends to decrease slightly.
- **R-squared ($R^2 = 0.042$):** Shows that baseline AQI explains only **4.2%** of the variance in rice yields across Indian states. The remaining 95.8% of variance is driven by crucial confounding factors.

Confounding Factors Acknowledgment (Correlation $\neq$ Causation)

This correlation does not prove that air pollution directly causes a decline in crop yields. The relationship is heavily shaped by unobserved variables that this dataset cannot account for:

1. **Monsoon/Rainfall:** Essential for rain-fed rice. Drought-prone states will have low yields regardless of air quality.
2. **Irrigation Infrastructure:** States with heavily managed canal/borewell systems (like Punjab and Haryana) achieve high yields despite severe air pollution.
3. **Soil Quality and Type:** Different soil profiles (alluvial vs black soil) have vastly different water and nutrient retention capabilities.
4. **Fertilizer & Seed Quality:** Usage of High-Yielding Varieties (HYV) and NPK

fertilizers strongly determines output, acting as a buffer against air quality stress.

Task C: One Plot to Rule Them All

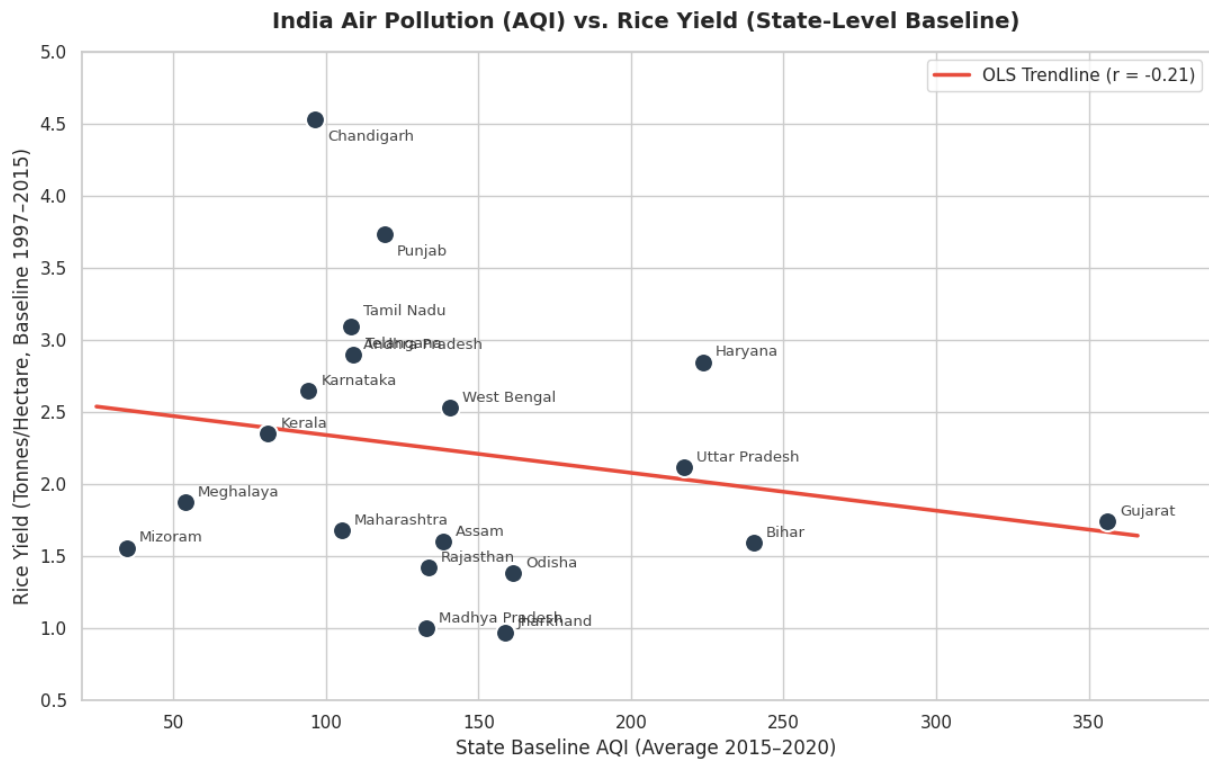
```
In [12]: plt.figure(figsize=(11, 7))

# Scatter plot of states
sns.scatterplot(data=merged_rice, x='AQI', y='Rice_Yield', s=150, color='#2c

# Add regression line
x_vals = np.linspace(merged_rice['AQI'].min() - 10, merged_rice['AQI'].max()
y_vals = slope * x_vals + intercept
plt.plot(x_vals, y_vals, color='#e74c3c', linestyle='-', linewidth=2.5, label=

# Annotate each state name with careful offsets
for idx, row in merged_rice.iterrows():
    offset_x = 4
    offset_y = 0.04
    if row['State_Name'] == 'Punjab':
        offset_y = -0.15
    elif row['State_Name'] == 'Chandigarh':
        offset_y = -0.15
    elif row['State_Name'] == 'Andhra Pradesh':
        offset_y = -0.15
    elif row['State_Name'] == 'Tamil Nadu':
        offset_y = 0.08
    plt.text(row['AQI'] + offset_x, row['Rice_Yield'] + offset_y, row['State

plt.title('India Air Pollution (AQI) vs. Rice Yield (State-Level Baseline)',
plt.xlabel('State Baseline AQI (Average 2015-2020)', fontsize=12)
plt.ylabel('Rice Yield (Tonnes/Hectare, Baseline 1997-2015)', fontsize=12)
plt.xlim(20, 390)
plt.ylim(0.5, 5.0)
plt.legend(fontsize=11, loc='upper right')
plt.tight_layout()
plt.show()
```



Why This Chart Matters

This scatter plot illustrates the negative relationship between a state's baseline air pollution (average AQI) and its agricultural productivity (Rice Yield). It reveals that states with cleaner air, such as Kerala and Telangana, achieve significantly higher yields than highly polluted states like Bihar and Jharkhand, although agricultural giants like Punjab are outliers due to heavy irrigation. This single chart highlights the tension between industrialization, poor air quality, and agricultural sustainability, underscoring that cleaner air is not just a health necessity but a key component of food security.